

Dong-A Univ. (ISPL)



동아대학교
DONG-A UNIVERSITY

Backpropagation (역전파) – 실습: **Vanishing Gradient Effect**

컴퓨터공학부
2024년 1학기 인공지능

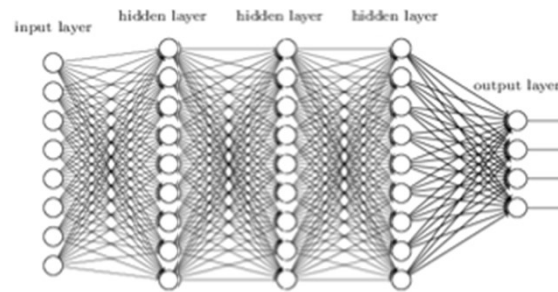
Contents

1. **Vanishing Gradient 현상이란?**
2. Vanishing Gradient 원인
3. Vanishing Gradient 해결법

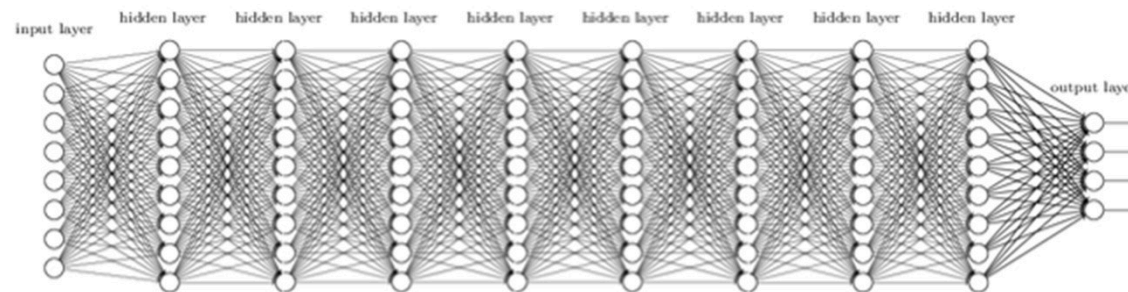


Vanishing Gradient

- 기울기 소실 (Vanishing gradient) 현상이란?
 - 일반적으로 신경망을 깊게 설계할수록 더 좋은 성능을 기대함



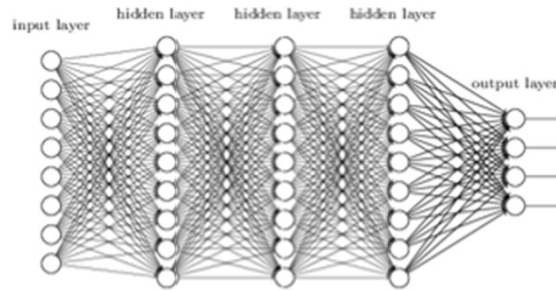
Neural Network



Deep Neural Network

Vanishing Gradient

- 기울기 소실 (Vanishing gradient) 현상이란?
 - 일반적으로 신경망을 깊게 설계할수록 더 좋은 성능을 기대함
 - 하지만, 깊은 신경망 설계 시 **Vanishing gradient** 현상 발생

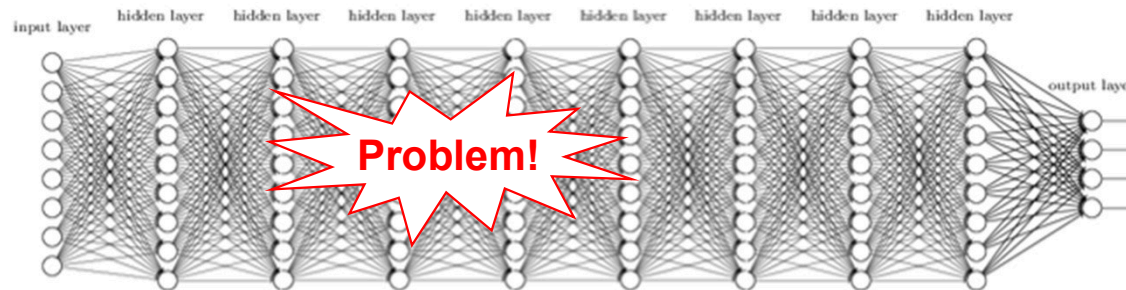


Neural Network

학습률(learning rate, step size)

$$w^{t+1} \leftarrow w^t - \mu \nabla L(w^t), \text{ where } \nabla L(w^t) \Delta w < 0$$

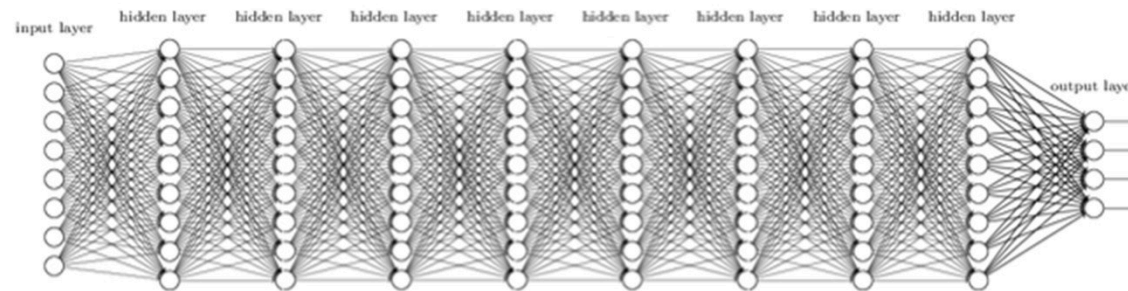
탐색 방향



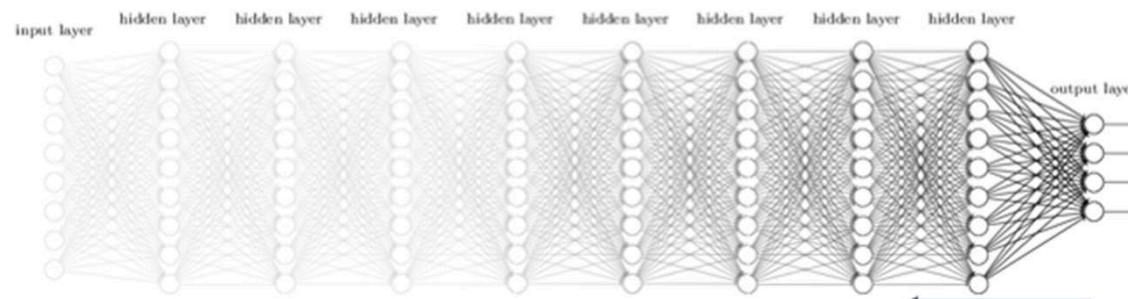
Deep Neural Network

Vanishing Gradient

- 기울기 소실 (Vanishing gradient) 현상이란?
 - Backpropagation 과정 시 입력 층에 가까워질 수록 기울기 값이 점점 0으로 수렴



Deep Neural Network



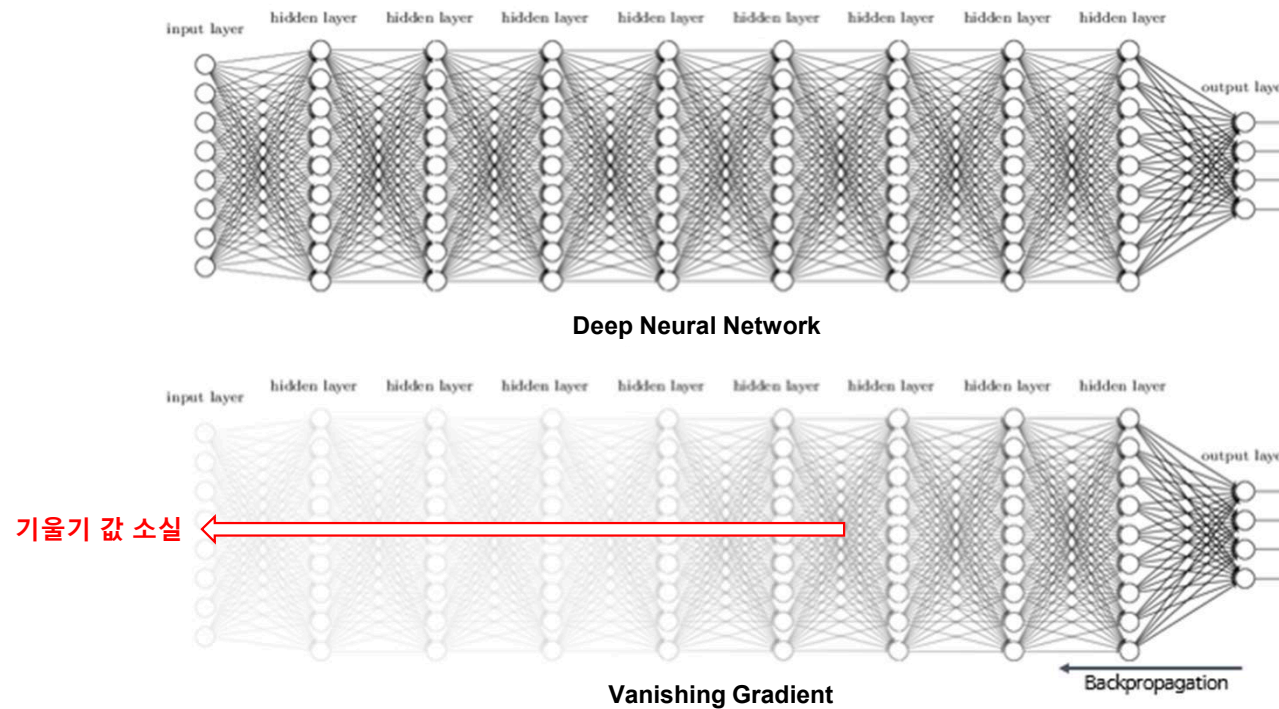
Vanishing Gradient

Backpropagation

Vanishing Gradient

기울기 소실 (Vanishing gradient) 현상이란?

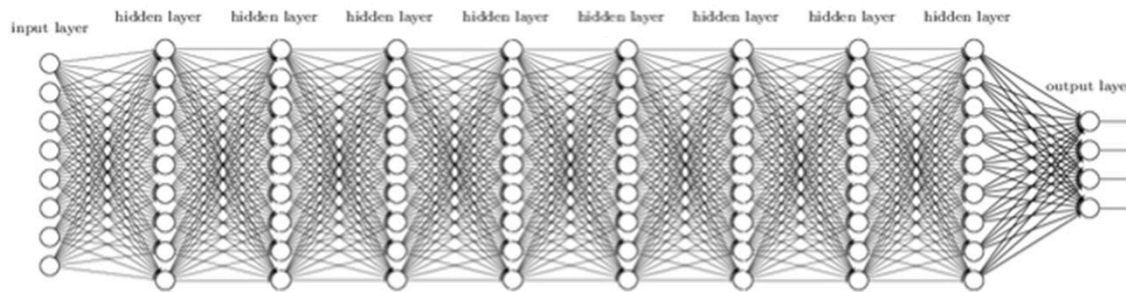
- Backpropagation 과정 시 입력 층에 가까워질 수록 기울기 값이 점점 0으로 수렴 → 기울기 값 소실



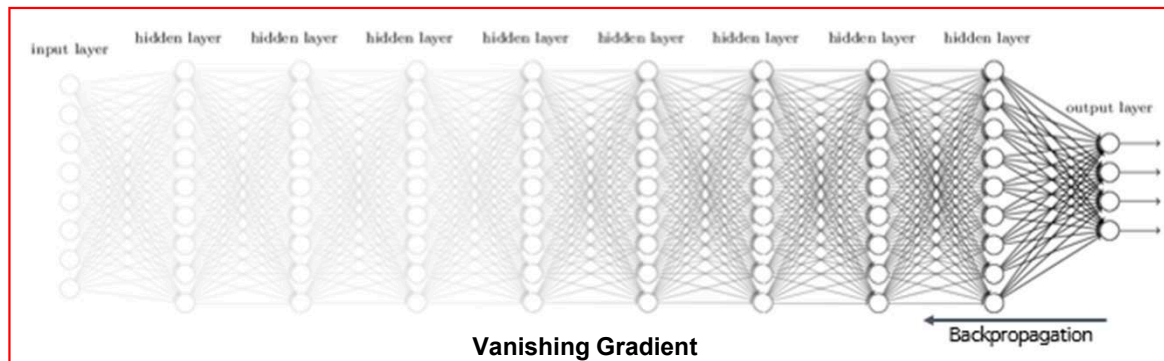
Vanishing Gradient

기울기 소실 (Vanishing gradient) 현상이란?

- Backpropagation 과정 시 입력 층에 가까워질 수록 기울기 값이 점점 0으로 수렴
- 소실된 기울기 값에 의해 신경망의 가중치를 업데이트 하는데 문제 발생



Deep Neural Network



Vanishing Gradient

$$W = W - \boxed{\eta} * \frac{\partial L}{\partial W}$$

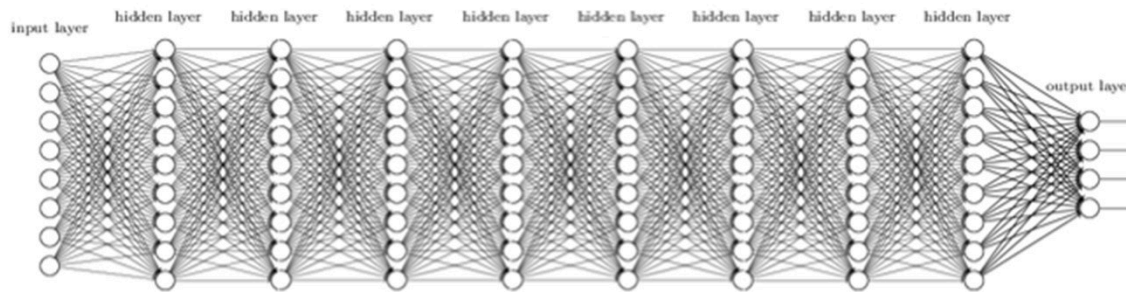
Learning rate Gradient

가중치 업데이트 수식

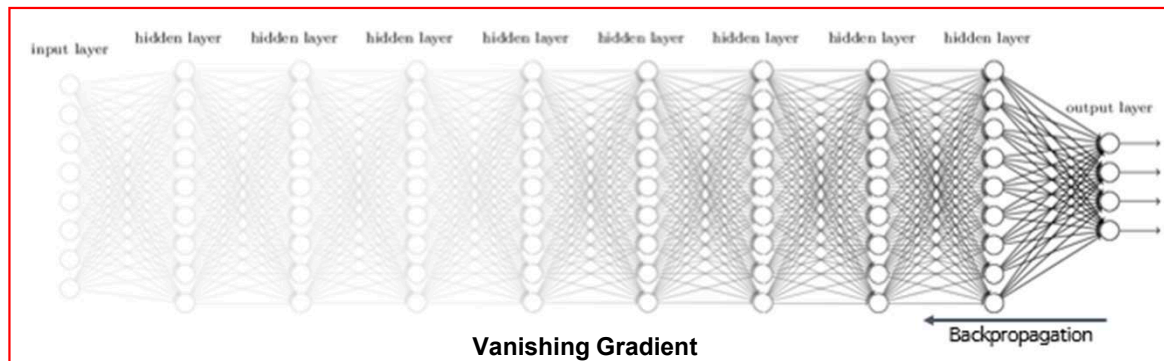
Vanishing Gradient

기울기 소실 (Vanishing gradient) 현상이란?

- Backpropagation 과정 시 입력 층에 가까워질 수록 기울기 값이 점점 0으로 수렴
- 소실된 기울기 값에 의해 신경망의 가중치를 업데이트 하는데 문제 발생 → 최적의 모델을 찾을 수 없음



Deep Neural Network



Vanishing Gradient

$$W = W - \boxed{\eta} * \frac{\partial L}{\partial W}$$

Learning rate Gradient

가중치 업데이트 수식

Contents

1. Vanishing Gradient 현상이란?
2. **Vanishing Gradient 원인**
3. Vanishing Gradient 해결법



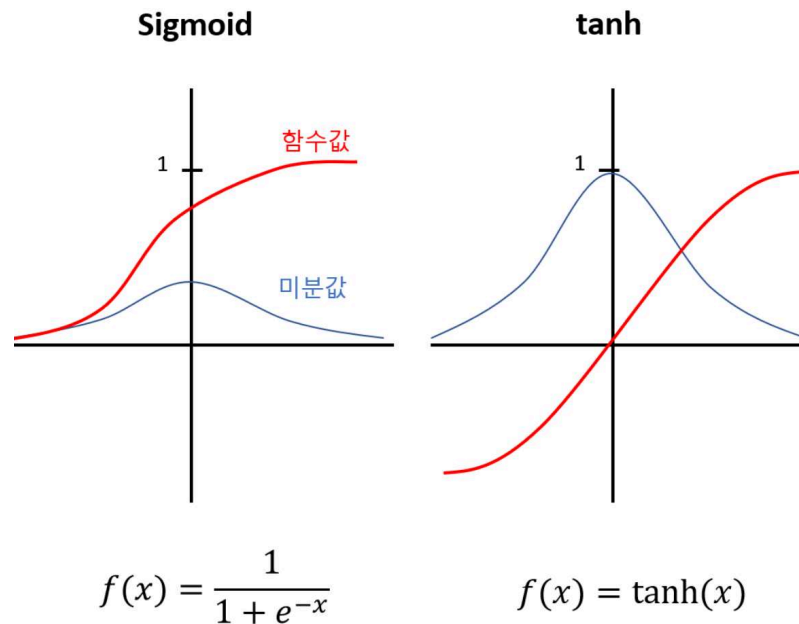
Vanishing Gradient

- 기울기 소실 (Vanishing gradient) 원인
 - (1) 활성화 함수의 미분 값의 최대치가 1보다 작은 경우
 - (2) 가중치 초기화가 제대로 되지 않은 경우

Vanishing Gradient

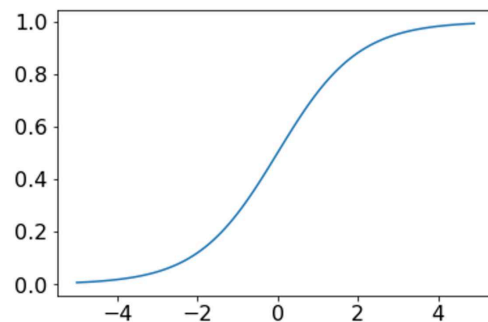
- 기울기 소실 (Vanishing gradient) 원인: 활성화 함수 (Activation function)
 - 활성화 함수의 미분 값의 최대치가 1보다 작은 경우 → Sigmoid, tanh (Hyperbolic tangent)

$$\begin{aligned}\frac{d}{dx} \text{sigmoid}(x) &= \frac{d}{dx} (1 + e^{-x})^{-1} \\&= (-1) \frac{1}{(1 + e^{-x})^2} \frac{d}{dx} (1 + e^{-x}) \\&= (-1) \frac{1}{(1 + e^{-x})^2} (0 + e^{-x}) \frac{d}{dx} (-x) \\&= (-1) \frac{1}{(1 + e^{-x})^2} e^{-x} (-1) \\&= \frac{e^{-x}}{(1 + e^{-x})^2} \\&= \frac{1 + e^{-x} - 1}{(1 + e^{-x})^2} \\&= \frac{(1 + e^{-x})}{(1 + e^{-x})^2} - \frac{1}{(1 + e^{-x})^2} \\&= \frac{1}{1 + e^{-x}} - \frac{1}{(1 + e^{-x})^2} \\&= \frac{1}{1 + e^{-x}} \left(1 - \frac{1}{1 + e^{-x}}\right) \\&= \text{sigmoid}(x)(1 - \text{sigmoid}(x))\end{aligned}$$



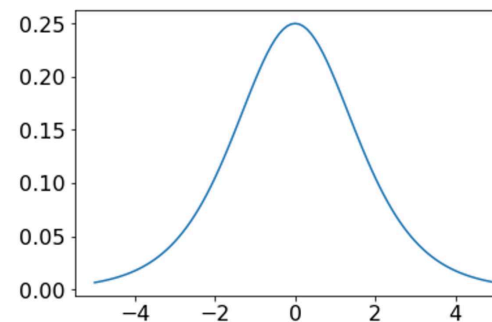
Vanishing Gradient

- 기울기 소실 (Vanishing gradient) 원인: 활성화 함수 (Activation function)
 - 활성화 함수의 미분 값의 최대치가 1보다 작은 경우 → Sigmoid, tanh (Hyperbolic tangent)
 - Sigmoid 함수의 경우 미분 값의 최대치가 0.25이므로 Vanishing gradient 현상 발생



$$\text{Sigmoid}(x) = \frac{1}{1+e^{-x}}$$

함수 값

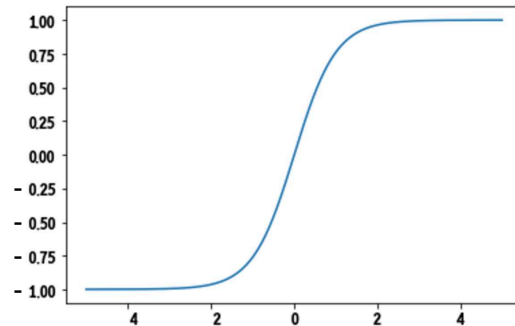


$$S'(x) = \frac{1}{1+e^{-x}} \left(1 - \frac{1}{1+e^{-x}} \right)$$

미분 값

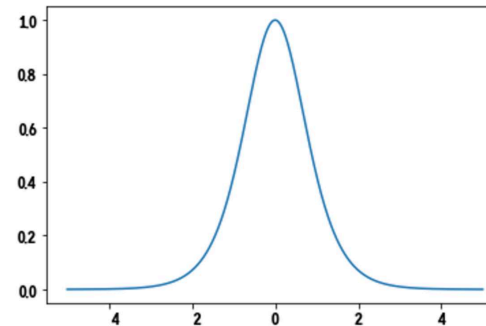
Vanishing Gradient

- 기울기 소실 (Vanishing gradient) 원인: 활성화 함수 (Activation function)
 - 활성화 함수의 미분 값의 최대치가 1보다 작은 경우 → Sigmoid, tanh (Hyperbolic tangent)
 - Sigmoid 함수의 경우 미분 값의 최대치가 0.25이므로 Vanishing gradient 현상 발생
 - Tanh 함수의 경우 미분 값이 여전히 1보다 작은 값을 가지므로 Vanishing gradient 현상 발생



$$\text{Tanh}(x) = \frac{e^{2x} + 1}{e^{2x} - 1}$$

함수 값



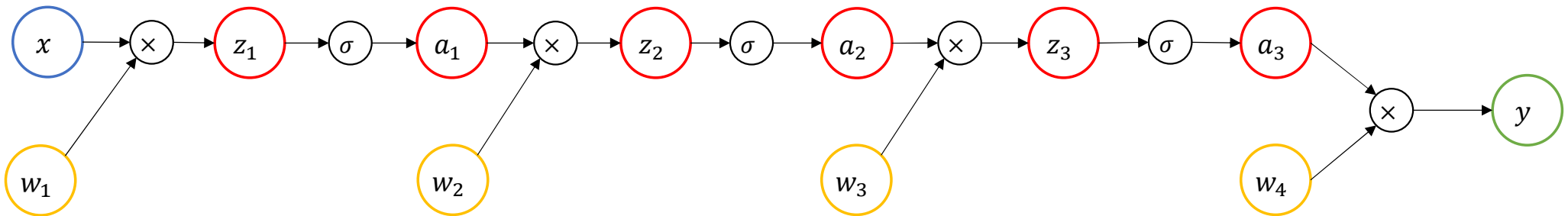
$$T'(x) = 1 - \tanh^2(x)$$

미분 값

Vanishing Gradient

- 기울기 소실 (Vanishing gradient) 원인: 활성화 함수 (Activation function)

- Example

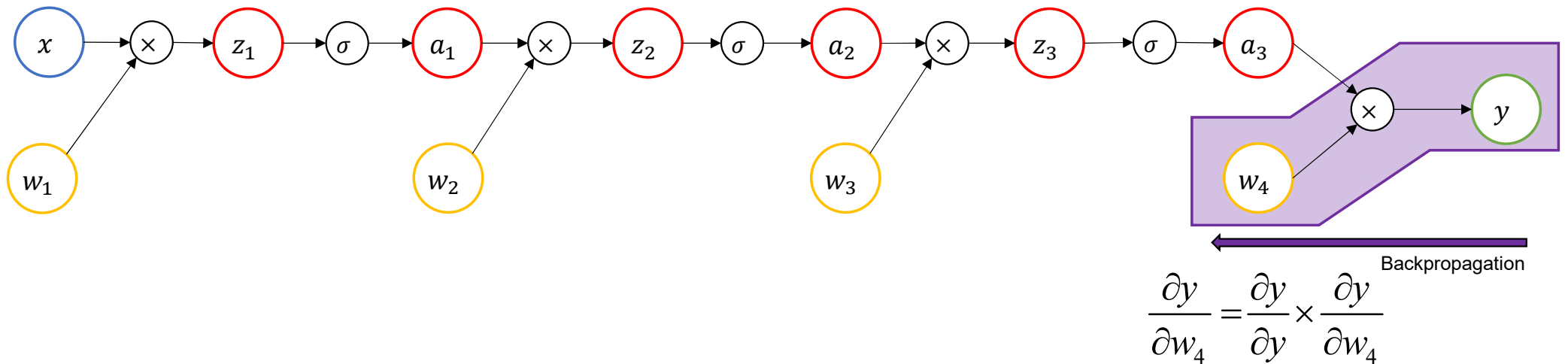


x : Input
 y : Output
 w_i : 가중치
 z_i : 은닉층의 입력 값
 a_i : 은닉층의 출력 값
 σ : Sigmoid function

Vanishing Gradient

- 기울기 소실 (Vanishing gradient) 원인: 활성화 함수 (Activation function)

- Example

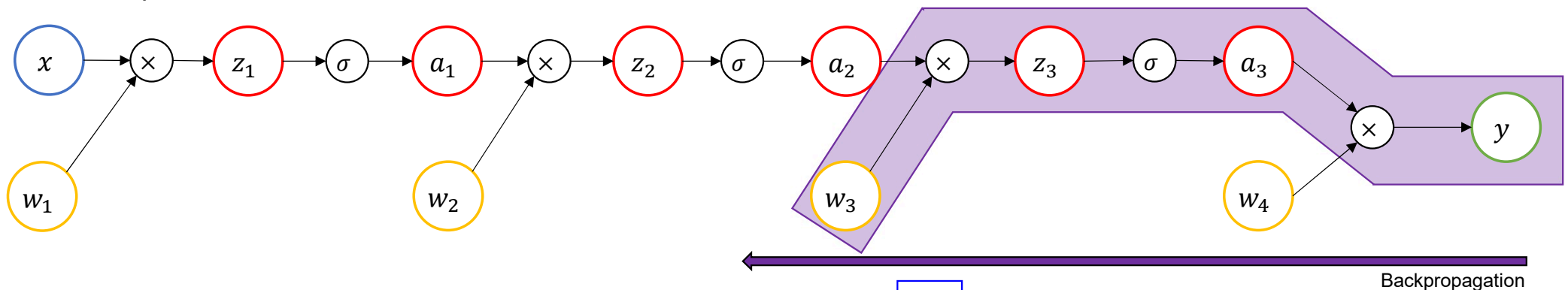


x : Input
 y : Output
 w_i : 가중치
 z_i : 은닉층의 입력 값
 a_i : 은닉층의 출력 값
 σ : Sigmoid function

Vanishing Gradient

기울기 소실 (Vanishing gradient) 원인: 활성화 함수 (Activation function)

Example



$$\frac{\partial y}{\partial w_3} = \frac{\partial y}{\partial y} \times \frac{\partial y}{\partial a_3} \times \boxed{\frac{\partial a_3}{\partial z_3}} \times \frac{\partial z_3}{\partial a_2}$$

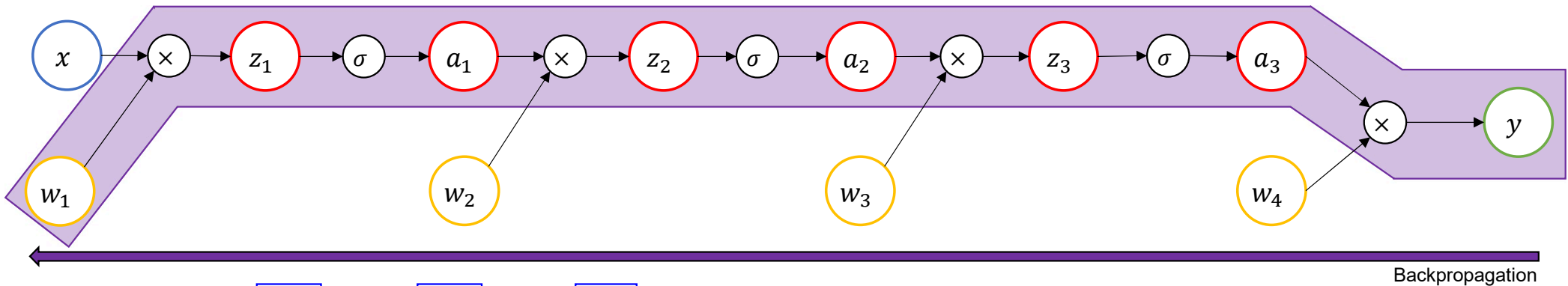
↓
Sigmoid function의 미분 값 < 1

x : Input
 y : Output
 w_i : 가중치
 z_i : 은닉층의 입력 값
 a_i : 은닉층의 출력 값
 σ : Sigmoid function

Vanishing Gradient

기울기 소실 (Vanishing gradient) 원인: 활성화 함수 (Activation function)

Example



$$\frac{\partial y}{\partial w_1} = \frac{\partial y}{\partial y} \times \frac{\partial y}{\partial a_3} \times \frac{\partial a_3}{\partial z_3} \times \frac{\partial z_3}{\partial a_2} \times \frac{\partial a_2}{\partial z_2} \times \frac{\partial z_2}{\partial a_1} \times \frac{\partial a_1}{\partial z_1} \times \frac{\partial z_1}{\partial w_1}$$

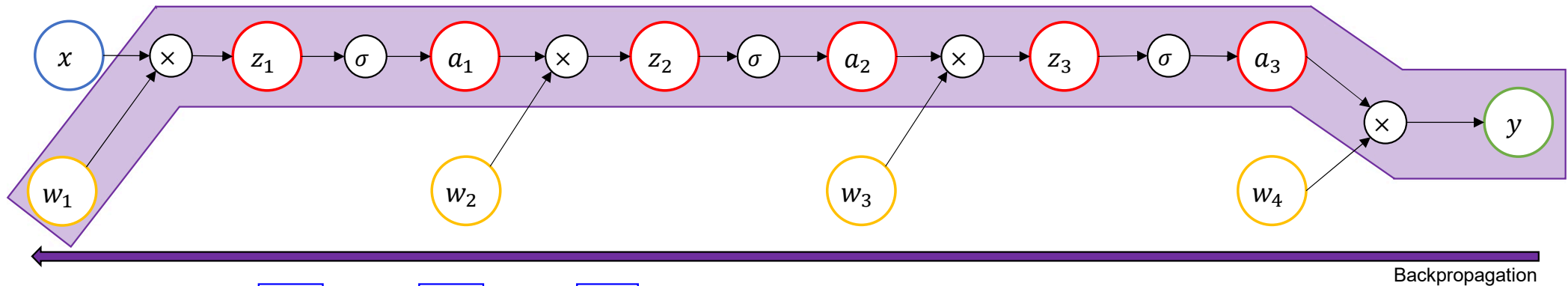
Sigmoid function의 미분 값 < 1

x : Input
 y : Output
 w_i : 가중치
 z_i : 은닉층의 입력 값
 a_i : 은닉층의 출력 값
 σ : Sigmoid function

Vanishing Gradient

기울기 소실 (Vanishing gradient) 원인: 활성화 함수 (Activation function)

Example



$$\frac{\partial y}{\partial w_1} = \frac{\partial y}{\partial y} \times \frac{\partial y}{\partial a_3} \times \frac{\partial a_3}{\partial z_3} \times \frac{\partial z_3}{\partial a_2} \times \frac{\partial a_2}{\partial z_2} \times \frac{\partial z_2}{\partial a_1} \times \frac{\partial a_1}{\partial z_1} \times \frac{\partial z_1}{\partial w_1}$$

- 가중치의 기울기를 구할 때 이전 층들의 미분 값이 사용
- 결론적으로, Backpropagation 시 입력 층에 가까워질 수록 **활성화 함수의 미분 값**에 의해 **기울기 소실 현상 발생**

x: Input
y: Output
w_i: 가중치
z_i: 은닉층의 입력 값
a_i: 은닉층의 출력 값
σ: Sigmoid function

Vanishing Gradient

- 기울기 소실 (Vanishing gradient) 원인: 가중치 초기화 (Weight initialization)
 - 가중치를 모두 0으로 초기화한 경우
 - 가중치를 평균이 0, 표준편차가 1인 정규분포로 초기화한 경우
 - 가중치를 평균이 0, 표준편차가 0.01인 정규분포로 초기화한 경우

Vanishing Gradient

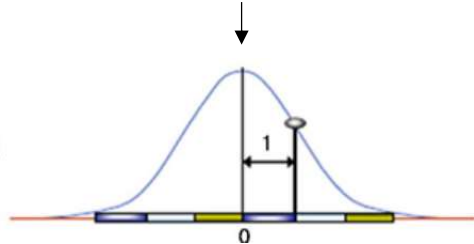
- 기울기 소실 (Vanishing gradient) 원인: 가중치 초기화 (Weight initialization)
 - 가중치를 모두 0으로 초기화한 경우
 - ✓ Backpropagation시 곱셈 과정에서 문제 → 신경망 학습 안됨

Vanishing Gradient

기울기 소실 (Vanishing gradient) 원인: 가중치 초기화 (Weight initialization)

- 가중치를 모두 0으로 초기화한 경우
- 가중치를 평균이 0, 표준편차가 1인 정규분포로 초기화한 경우
 - ✓ 출력 값들이 -1과 1로 치우침 → **Vanishing gradient 문제 발생**

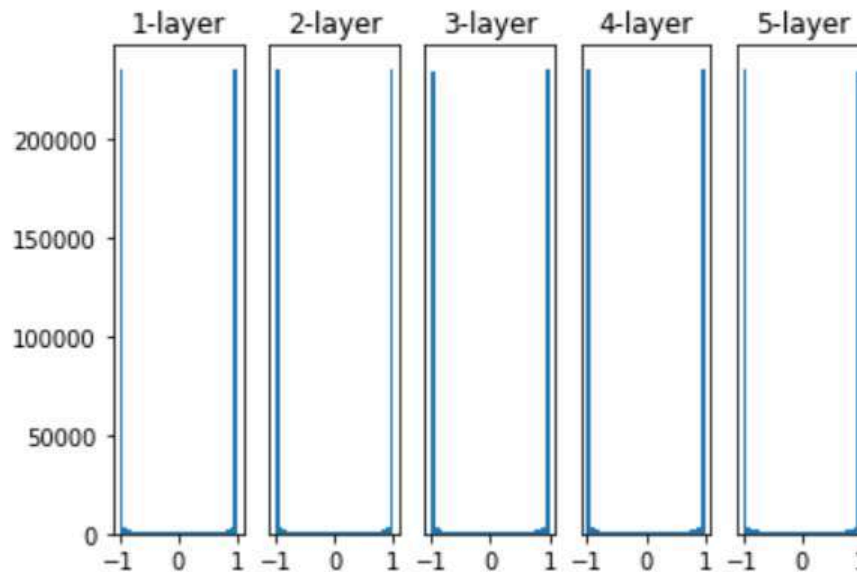
$$y = \tanh(W \times x + b)$$



표준정규분포

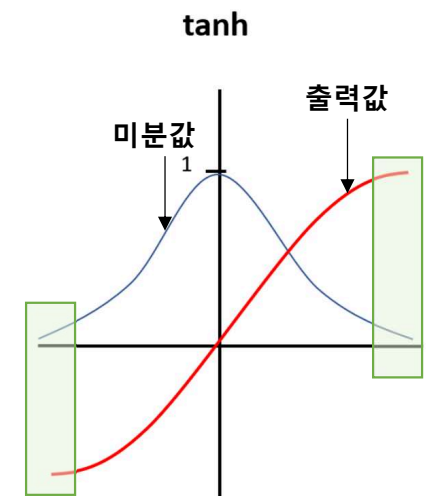
Weight 값이 커짐으로 인해 **출력 값 증가**

Activation Function: **tanh**



각층의 활성화 값 분포

21/49



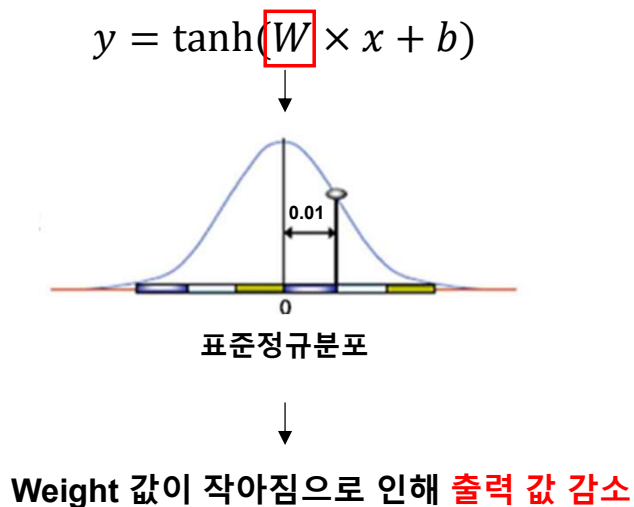
$$f(x) = \tanh(x)$$

Vanishing Gradient

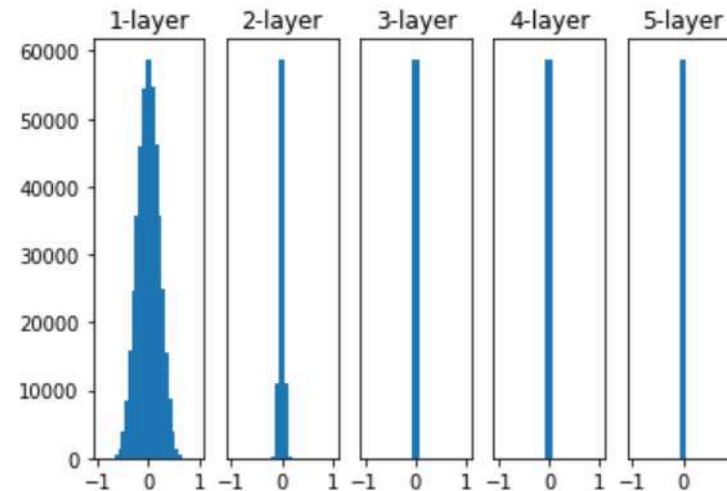
기울기 소실 (Vanishing gradient) 원인: 가중치 초기화 (Weight initialization)

- 가중치를 모두 0으로 초기화한 경우
- 가중치를 평균이 0, 표준편차가 1인 정규분포로 초기화한 경우
- 가중치를 평균이 0, 표준편차가 0.01인 정규분포로 초기화한 경우

✓ Vanishing gradient는 해결되지만 출력 값이 0으로 치우치는 현상 발생 → **신경망 학습 안됨**

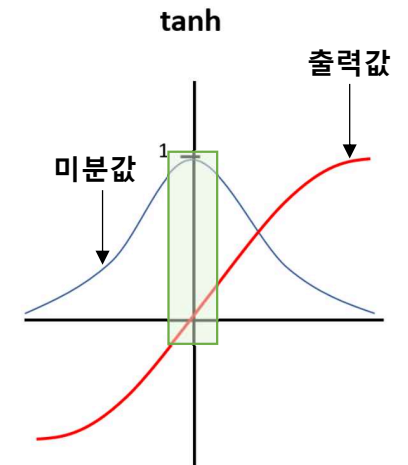


Activation Function: **tanh**



각층의 활성화 값 분포

22/49



$$f(x) = \tanh(x)$$

Contents

1. Vanishing Gradient 현상이란?
2. Vanishing Gradient 원인
3. **Vanishing Gradient 해결법**



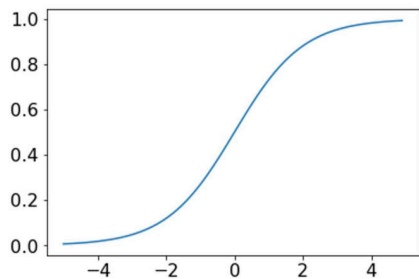
Vanishing Gradient

- 기울기 소실 (Vanishing gradient) 해결법
 - 활성화 함수 변경
 - 가중치 초기화 설정

Vanishing Gradient

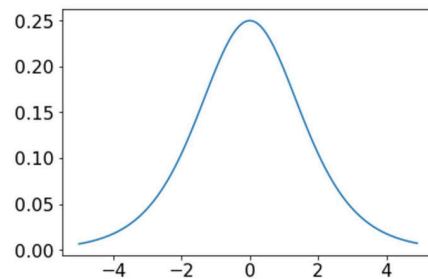
기울기 소실 (Vanishing gradient) 해결법: 활성화 함수 변경

- Sigmoid, tanh → ReLU, PReLU, Leaky ReLU, etc...
- 미분 값의 **최대치가 1보다 큰 활성화 함수**를 사용하여 기울기 소실 문제 해결 가능



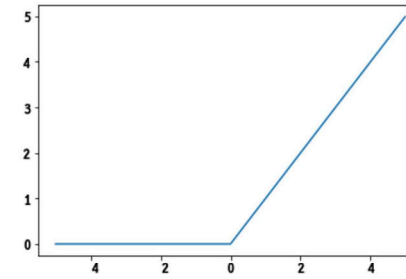
$$\text{Sigmoid}(x) = \frac{1}{1+e^{-x}}$$

함수 값



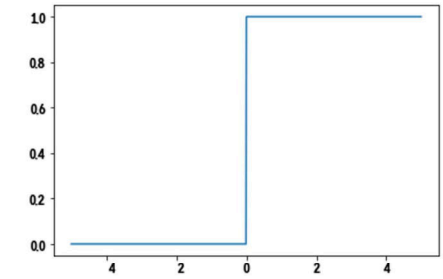
$$S'(x) = \frac{1}{1+e^{-x}} \left(1 - \frac{1}{1+e^{-x}} \right)$$

미분 값



$$\text{ReLU}(x) = \max(0, x)$$

함수 값



$$R'(y) = \begin{cases} 1 & (x \geq 0) \\ 0 & (x < 0) \end{cases}$$

미분 값

Vanishing Gradient

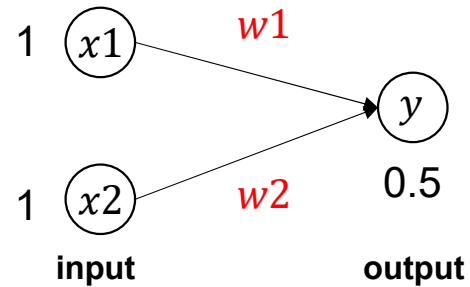
- 기울기 소실 (Vanishing gradient) 해결법: 가중치 초기화 (Weight initialization)
 - Xavier, He initialization 등을 사용하여 가중치 값을 초기화 → 입력 Node 개수가 wight를 초기화 하는데 영향을 줌

Vanishing Gradient

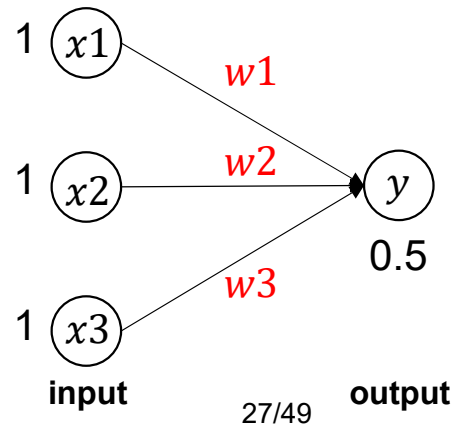
기울기 소실 (Vanishing gradient) 해결법: 가중치 초기화 (Weight initialization)

- Xavier, He initialization 등을 사용하여 가중치 값을 초기화 → 입력 Node 개수가 wight를 초기화 하는데 영향을 줌

입력 노드가 2개인 경우 weight 예시)



입력 노드가 3개인 경우 weight 예시)

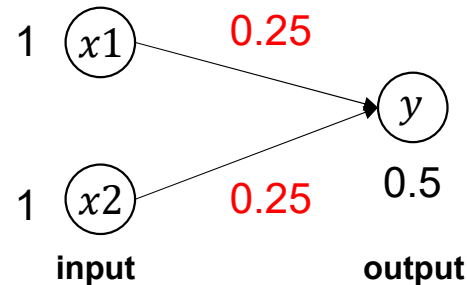


Vanishing Gradient

기울기 소실 (Vanishing gradient) 해결법: 가중치 초기화 (Weight initialization)

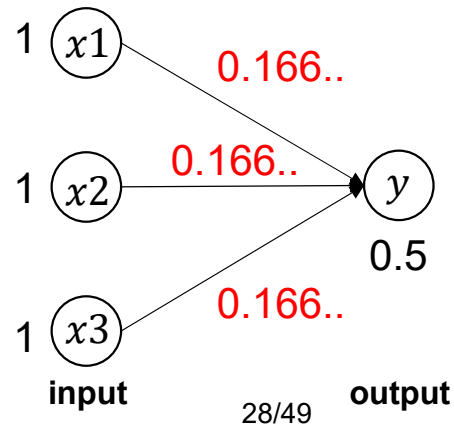
- Xavier, He initialization 등을 사용하여 가중치 값을 초기화 → 입력 node 개수에 따라 weight를 초기화 해줘야 함

입력 노드가 2개인 경우 weight 예시)



$$y = (x_1 * w_1) + (x_2 * w_2)$$
$$0.5 = (1 * 0.25) + (1 * 0.25)$$

입력 노드가 3개인 경우 weight 예시)



$$y = (x_1 * w_1) + (x_2 * w_2) + (x_3 * w_3)$$
$$0.5 = (1 * 0.166) + (1 * 0.166) + (1 * 0.166)$$

Vanishing Gradient

■ 기울기 소실 (Vanishing gradient) 해결법: 가중치 초기화 (Weight initialization)

- Xavier, He initialization 등을 사용하여 가중치 값을 초기화
 - ✓ Xavier initialization: 이전 층과 다음 층의 노드의 수를 반영

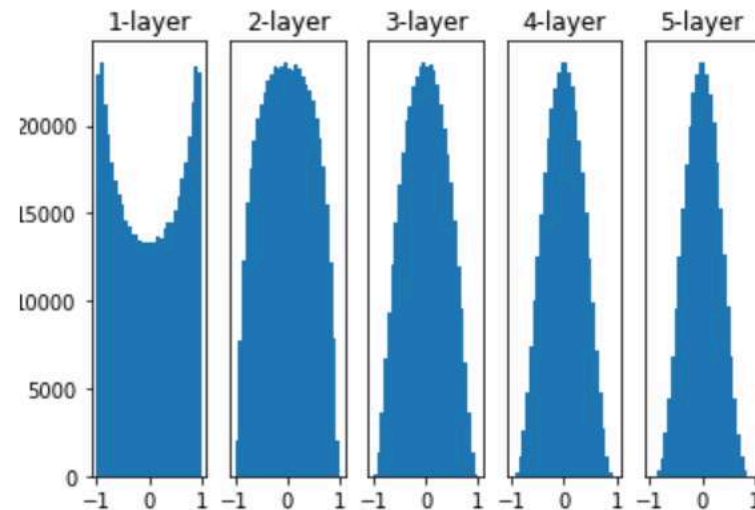
➤ Xavier Normal Initialization

$$W \sim N(0, Var(W))$$

$$Var(W) = \sqrt{\frac{2}{n_{in} + n_{out}}}$$

n_{in} : 이전 layer (input)의 노드의 수
 n_{out} : 다음 layer의 노드의 수

Activation Function: **tanh**



Vanishing Gradient

기울기 소실 (Vanishing gradient) 해결법: 가중치 초기화 (Weight initialization)

- Xavier, He initialization 등을 사용하여 가중치 값을 초기화
 - ✓ Xavier initialization: 이전 층과 다음 층의 노드의 수를 반영 → ReLU 활성화 함수를 사용 시 문제 발생

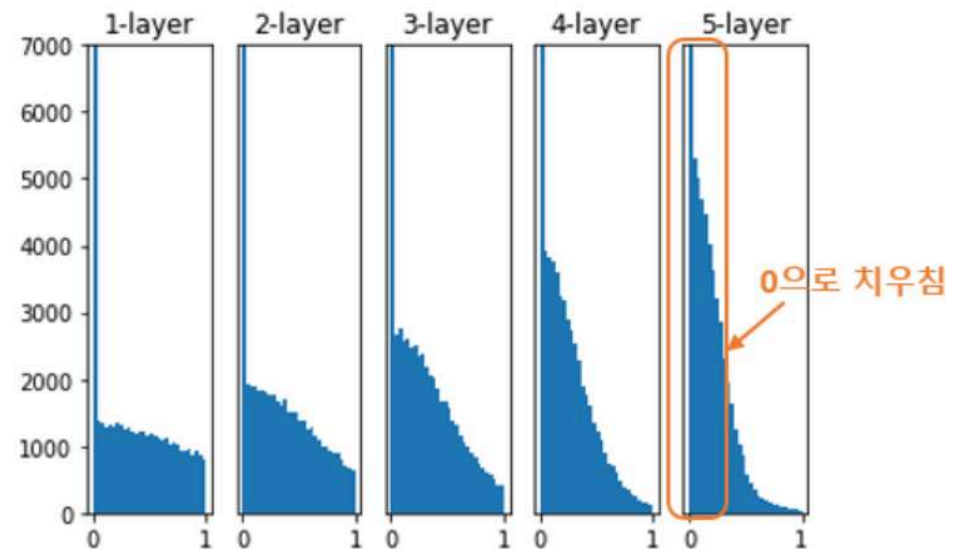
➤ Xavier Normal Initialization

$$W \sim N(0, Var(W))$$

$$Var(W) = \sqrt{\frac{2}{n_{in} + n_{out}}}$$

n_{in} : 이전 layer (input)의 노드의 수
 n_{out} : 다음 layer의 노드의 수

Activation Function: **ReLU**



Vanishing Gradient

기울기 소실 (Vanishing gradient) 해결법: 가중치 초기화 (Weight initialization)

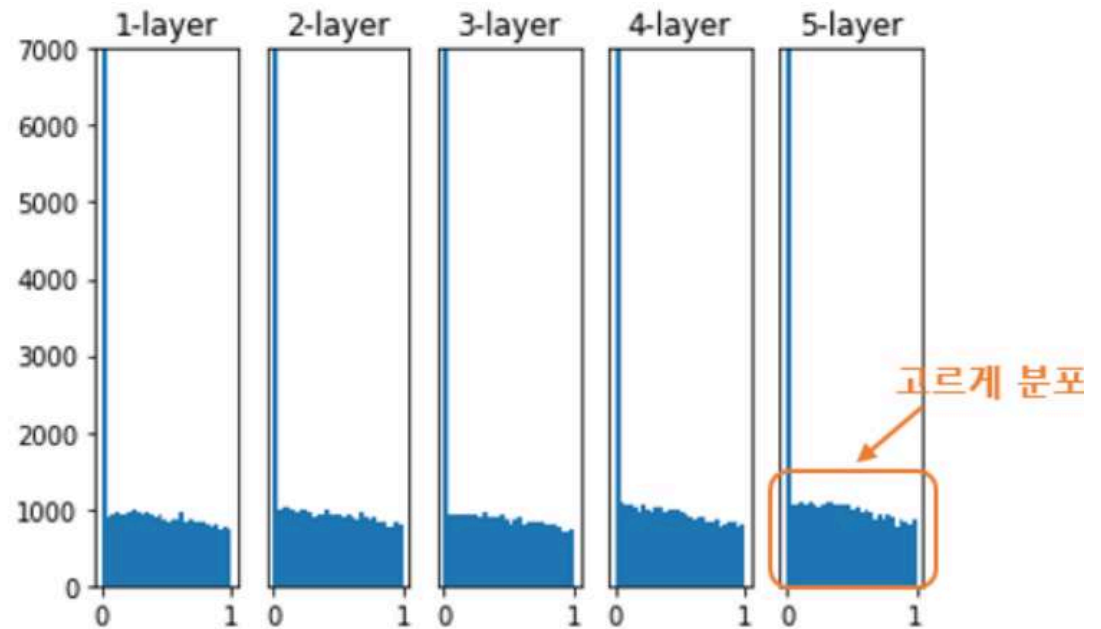
- Xavier, He initialization 등을 사용하여 가중치 값을 초기화
 - ✓ Xavier initialization: 이전 층과 다음 층의 노드의 수를 반영
 - ✓ He initialization: 이전 층의 노드 수만 반영 → ReLU함수에 적합한 초기화 방법

➤ He Normal Initialization

$$W \sim N(0, Var(W))$$

$$Var(W) = \sqrt{\frac{2}{n_{in}}}$$

n_{in} : 이전 layer (input)의 노드의 수

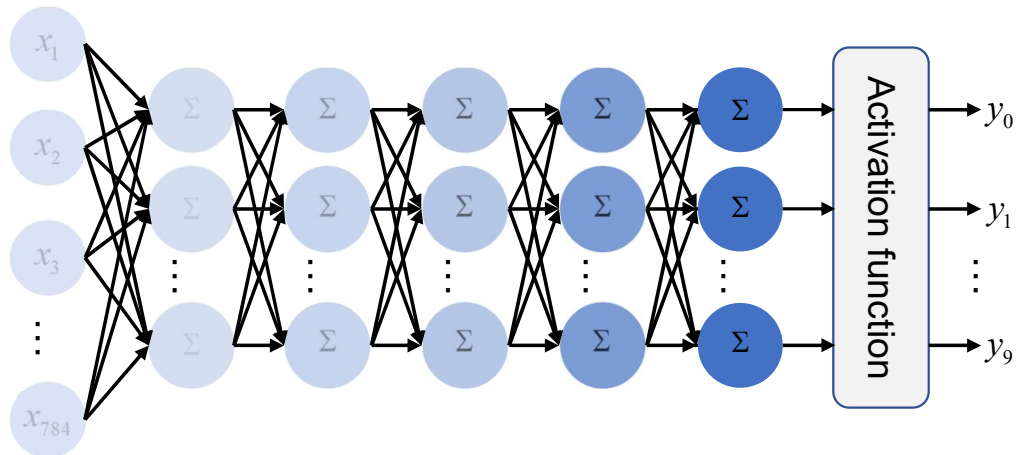


<실 습> - 오버피팅 -

Overfitting 문제 실습

Overfitting 이 주로 일어나는 경우

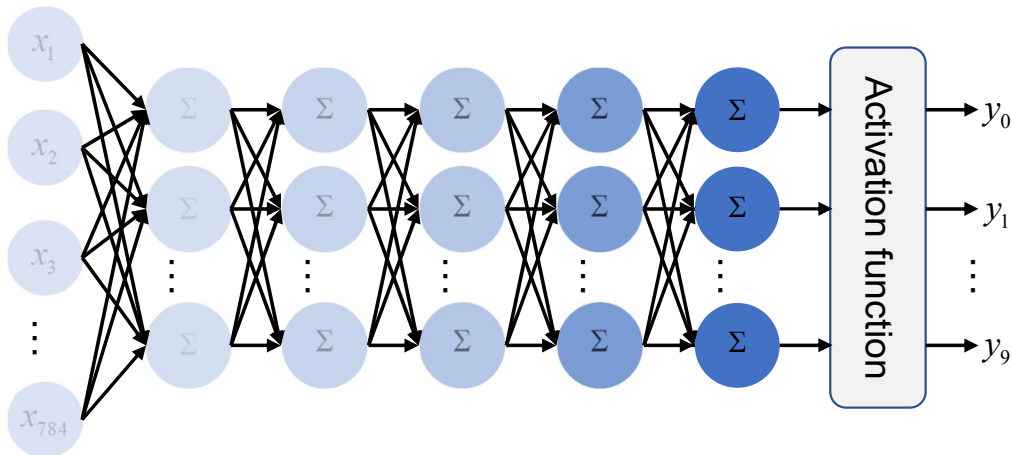
- 매개변수가 많은 표현력이 높은 모델
- 훈련데이터가 적음



Vanishing Gradient 문제 실습

- Vanishing Gradient 문제 확인: 깊은 신경망 설계

Vanishing Gradient 실습



- ✓ Layer1 (Input: 784, Out: 100)
- ✓ Layer2 (Input: 100, Out: 100)
- ✓ Layer3 (Input: 100, Out: 100)
- ✓ Layer4 (Input: 100, Out: 100)
- ✓ Layer5 (Input: 100, Out: 10)
- ✓ Activation function: **Sigmoid**
- ✓ Loss function: Cross Entropy

Vanishing Gradient 문제 실습

▪ Vanishing Gradient 문제 확인: 깊은 신경망 설계

• (1) MLP 모델 정의

▼ Multi-layer Perceptron 모델 정의: 5-layer

```
[13] 1 class FMLP(nn.Module): # 5-layer → Network 이름 변경
      2     def __init__(self):
      3         super(FMLP, self).__init__()
      4         self.fc1 = nn.Linear(in_features=784, out_features=100)
      5         self.fc2 = nn.Linear(in_features=100, out_features=100)
      6         self.fc3 = nn.Linear(in_features=100, out_features=100)
      7         self.fc4 = nn.Linear(in_features=100, out_features=100)
      8         self.fc5 = nn.Linear(in_features=100, out_features=10)
      9         self.sigmoid = nn.Sigmoid()
     10
     11     def forward(self, x):
     12         x = x.view(-1, 28*28)
     13         y = self.sigmoid(self.fc1(x))
     14         y = self.sigmoid(self.fc2(y))
     15         y = self.sigmoid(self.fc3(y))
     16         y = self.sigmoid(self.fc4(y))
     17         y = self.fc5(y)
     18         return y
```

→ Layer 5개 추가 (Input, Output node 개수 주의)

Vanishing Gradient 문제 실습

- **Vanishing Gradient 문제 확인: 깊은 신경망 설계**

- (2) Hyper-parameter 지정 (2-layer 환경이랑 동일)

▼ Hyper-parameter 지정

```
[14] 1 batch_size = 100
      2 learning_rate = 0.1
      3 training_epochs = 15
      4 loss_function = nn.CrossEntropyLoss()
      5 network = FMLP()
      6 optimizer = torch.optim.SGD(network.parameters(), lr = learning_rate)
      7
      8 data_loader = DataLoader(dataset = mnist_train,
      9                        batch_size = batch_size,
     10                        shuffle = True,
     11                        drop_last = True)
```

→ Network 이름 변경 주의

Vanishing Gradient 문제 실습

- **Vanishing Gradient 문제 확인: 깊은 신경망 설계**
 - (3) MLP 학습을 위한 반복문 선언 (2-layer 환경이랑 동일)

Network Training 진행

```
[15] 1 for epoch in range(training_epochs):
      2     avg_cost = 0
      3     total_batch = len(data_loader)
      4
      5     for img, label in data_loader:
      6         pred = network(img)
      7
      8         loss = loss_function(pred, label)
      9         optimizer.zero_grad()
     10         loss.backward()
     11         optimizer.step()
     12
     13         avg_cost += loss / total_batch
     14
     15     print('Epoch: %d Loss = %f' %(epoch+1, avg_cost))
     16 print('Learning finished')
```

Vanishing Gradient 문제 실습

■ Vanishing Gradient 문제 확인: 깊은 신경망 설계

- (3) MLP 학습을 위한 반복문 선언 (2-layer 환경이랑 동일) → 결과 확인

Network Training 진행

```
[15] 1 for epoch in range(training_epochs):
      2     avg_cost = 0
      3     total_batch = len(data_loader)
      4
      5     for img, label in data_loader:
      6         pred = network(img)
      7
      8         loss = loss_function(pred, label)
      9         optimizer.zero_grad()
     10         loss.backward()
     11         optimizer.step()
     12
     13         avg_cost += loss / total_batch
     14
     15     print('Epoch: %d Loss = %f' %(epoch+1, avg_cost))
     16 print('Learning finished')
```

Vanishing Gradient 문제 발생!

```
Epoch: 1 Loss = 2.308455
Epoch: 2 Loss = 2.307792
Epoch: 3 Loss = 2.306702
Epoch: 4 Loss = 2.306380
Epoch: 5 Loss = 2.305326
Epoch: 6 Loss = 2.305197
Epoch: 7 Loss = 2.304592
Epoch: 8 Loss = 2.304827
Epoch: 9 Loss = 2.304305
Epoch: 10 Loss = 2.303926
Epoch: 11 Loss = 2.303735
Epoch: 12 Loss = 2.303237
Epoch: 13 Loss = 2.303161
Epoch: 14 Loss = 2.302957
Epoch: 15 Loss = 2.302973
Learning finished
```

Vanishing Gradient 문제 실습

▪ Vanishing Gradient 문제 확인: 깊은 신경망 설계

- (4) 학습 완료된 Weight parameter 저장 및 확인
- (5) MNIST Test dataset 분류 성능 확인

```
[16] 1 torch.save(network.state_dict(), parameterPath+"fmlp_mnist.pth")  
  
[17] 1 new_network = FMLP()  
     2 new_network.load_state_dict(torch.load(parameterPath+"fmlp_mnist.pth"))  
  
<All keys matched successfully>
```

```
img_test = mnist_test.data.float()  
label_test = mnist_test.targets  
  
prediction = network(img_test) # 전체 test data를 한번에 계산  
  
correct_prediction = torch.argmax(prediction, 1) == label_test  
accuracy = correct_prediction.float().mean()  
print("Accuracy:", accuracy.item())  
  
Accuracy: 0.11349999904632568
```

정답률: 11.3%

Vanishing Gradient 문제 실습

Vanishing Gradient 문제 확인: 깊은 신경망 설계

- (6) 예측 결과 값 및 정답 이미지 확인

```
✓ [19] 1 first_data = mnist_test.data[0] # Test dataset 중 첫번째 Image 추출
0초 2 with torch.no_grad():
3     prediction = network(first_data.view(-1, 784).float())
4     print(prediction)

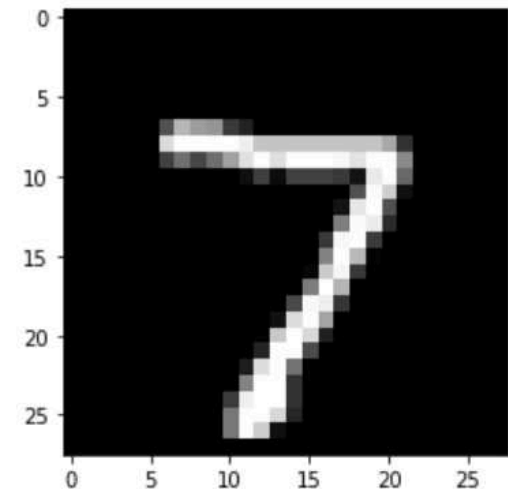
tensor([[ -0.0251,  0.1328, -0.0739, -0.0059, -0.1392,  0.0033, -0.1058,  0.0364,
          -0.1468, -0.0306]])

✓ [20] 1 prediction_num = torch.argmax(prediction) # 예측 점수가 가장 높은 숫자 가져오기
0초 2 print("예측 값은 %d 입니다." %(prediction_num))

예측 값은 1 입니다.
```

1로 예측하였지만 실제 결과는 7

```
[21] 1 import matplotlib.pyplot as plt
2     plt.imshow(first_data, cmap='gray')
3     plt.show()
```



Vanishing Gradient 문제 실습

■ Vanishing Gradient 문제 해결(1): 활성화 함수 변경

- (1) MLP 모델 재정의
 - Sigmoid 함수 → ReLU 함수

```
[22] 1 class FMLP(nn.Module): # 5-layer
      2     def __init__(self):
      3         super(FMLP, self).__init__()
      4         self.fc1 = nn.Linear(in_features=784, out_features=100)
      5         self.fc2 = nn.Linear(in_features=100, out_features=100)
      6         self.fc3 = nn.Linear(in_features=100, out_features=100)
      7         self.fc4 = nn.Linear(in_features=100, out_features=100)
      8         self.fc5 = nn.Linear(in_features=100, out_features=10)
      9         self.relu = nn.ReLU()
     10
     11     def forward(self, x):
     12         x = x.view(-1, 28*28)
     13         y = self.relu(self.fc1(x))
     14         y = self.relu(self.fc2(y))
     15         y = self.relu(self.fc3(y))
     16         y = self.relu(self.fc4(y))
     17         y = self.fc5(y)
     18         return y
```

→ ReLU 함수로 변경

Vanishing Gradient 문제 실습

■ Vanishing Gradient 문제 해결(1): 활성화 함수 변경

- (2) Hyper-parameter 지정 및 Training 진행

```
[23] 1 batch_size = 100
      2 learning_rate = 0.1
      3 training_epochs = 15
      4 loss_function = nn.CrossEntropyLoss()
      5 network = FMLP()
      6 optimizer = torch.optim.SGD(network.parameters(), lr = learning_rate)
      7
      8 data_loader = DataLoader(dataset = mnist_train,
      9                       batch_size = batch_size,
     10                       shuffle = True,
     11                       drop_last = True)
```

```
[24] 1 for epoch in range(training_epochs):
      2     avg_cost = 0
      3     total_batch = len(data_loader)
      4
      5     for img, label in data_loader:
      6         pred = network(img)
      7
      8         loss = loss_function(pred, label)
      9         optimizer.zero_grad()
     10         loss.backward()
     11         optimizer.step()
     12
     13         avg_cost += loss / total_batch
     14
     15     print('Epoch: %d Loss = %f' %(epoch+1, avg_cost))
     16 print('Learning finished')
```

Vanishing Gradient 문제 실습

■ Vanishing Gradient 문제 해결(1): 활성화 함수 변경

• (3) Network Training 결과 확인

```
[24] 1 for epoch in range(training_epochs):
      2     avg_cost = 0
      3     total_batch = len(data_loader)
      4
      5     for img, label in data_loader:
      6         pred = network(img)
      7
      8         loss = loss_function(pred, label)
      9         optimizer.zero_grad()
     10         loss.backward()
     11         optimizer.step()
     12
     13         avg_cost += loss / total_batch
     14
     15     print('Epoch: %d Loss = %f' %(epoch+1, avg_cost))
     16 print('Learning finished')
```

Vanishing Gradient 문제 해결



Epoch: 1	Loss = 1.162081
Epoch: 2	Loss = 0.252855
Epoch: 3	Loss = 0.153998
Epoch: 4	Loss = 0.117303
Epoch: 5	Loss = 0.094601
Epoch: 6	Loss = 0.077537
Epoch: 7	Loss = 0.067494
Epoch: 8	Loss = 0.056709
Epoch: 9	Loss = 0.049883
Epoch: 10	Loss = 0.042000
Epoch: 11	Loss = 0.037052
Epoch: 12	Loss = 0.031780
Epoch: 13	Loss = 0.027287
Epoch: 14	Loss = 0.024723
Epoch: 15	Loss = 0.022295
Learning finished	

Vanishing Gradient 문제 실습

■ Vanishing Gradient 문제 해결(1): 활성화 함수 변경

- (4) Weight parameter 저장 및 불러오기
- (5) MNIST Test dataset 분류 성능 확인

```
[26] 1 torch.save(network.state_dict(), parameterPath+"fmlp_mnist.pth")  
[27] 1 new_network = FMLP()  
      2 new_network.load_state_dict(torch.load(parameterPath+"fmlp_mnist.pth"))  
  
<All keys matched successfully>
```

```
[28] 1 with torch.no_grad(): # test에서는 기울기 계산 제외  
      2  
      3 img_test = mnist_test.data.float()  
      4 label_test = mnist_test.targets  
      5  
      6 prediction = network(img_test) # 전체 test data를 한번에 계산  
      7  
      8 correct_prediction = torch.argmax(prediction, 1) == label_test  
      9 accuracy = correct_prediction.float().mean()  
     10 print("Accuracy:", accuracy.item())
```

Accuracy: 0.9715999960899353

정답률: 97.1%

Vanishing Gradient 문제 실습

■ Vanishing Gradient 문제 해결(2): 가중치 초기화 사용

- (1) MLP 모델 재정의
 - Sigmoid 함수 사용
 - Xavier normal initialization 사용

Sigmoid 함수 정의 ←

각 Layer마다 Xavier 초기화 적용 ←

```
1 class FMLP(nn.Module): # 5-layer
2     def __init__(self):
3         super(FMLP, self).__init__()
4         self.fc1 = nn.Linear(in_features=784, out_features=100)
5         self.fc2 = nn.Linear(in_features=100, out_features=100)
6         self.fc3 = nn.Linear(in_features=100, out_features=100)
7         self.fc4 = nn.Linear(in_features=100, out_features=100)
8         self.fc5 = nn.Linear(in_features=100, out_features=10)
9         self.sigmoid = nn.Sigmoid()
10
11         torch.nn.init.xavier_normal_(self.fc1.weight.data)
12         torch.nn.init.xavier_normal_(self.fc2.weight.data)
13         torch.nn.init.xavier_normal_(self.fc3.weight.data)
14         torch.nn.init.xavier_normal_(self.fc4.weight.data)
15         torch.nn.init.xavier_normal_(self.fc5.weight.data)
16
17     def forward(self, x):
18         x = x.view(-1, 28*28)
19         y = self.sigmoid(self.fc1(x))
20         y = self.sigmoid(self.fc2(y))
21         y = self.sigmoid(self.fc3(y))
22         y = self.sigmoid(self.fc4(y))
23         y = self.fc5(y)
24         return y
```

Vanishing Gradient 문제 실습

▪ Vanishing Gradient 문제 해결(2): 가중치 초기화 사용

- (2) Hyper-parameter 지정 및 Training 진행

```
[23] 1 batch_size = 100
      2 learning_rate = 0.1
      3 training_epochs = 15
      4 loss_function = nn.CrossEntropyLoss()
      5 network = FMLP()
      6 optimizer = torch.optim.SGD(network.parameters(), lr = learning_rate)
      7
      8 data_loader = DataLoader(dataset = mnist_train,
      9                       batch_size = batch_size,
     10                       shuffle = True,
     11                       drop_last = True)
```

```
[24] 1 for epoch in range(training_epochs):
      2     avg_cost = 0
      3     total_batch = len(data_loader)
      4
      5     for img, label in data_loader:
      6         pred = network(img)
      7
      8         loss = loss_function(pred, label)
      9         optimizer.zero_grad()
     10         loss.backward()
     11         optimizer.step()
     12
     13         avg_cost += loss / total_batch
     14
     15     print('Epoch: %d Loss = %f' %(epoch+1, avg_cost))
     16 print('Learning finished')
```

Vanishing Gradient 문제 실습

■ Vanishing Gradient 문제 해결(2): 가중치 초기화 사용

• (3) Network Training 결과 확인

```
[24] 1 for epoch in range(training_epochs):
      2     avg_cost = 0
      3     total_batch = len(data_loader)
      4
      5     for img, label in data_loader:
      6         pred = network(img)
      7
      8         loss = loss_function(pred, label)
      9         optimizer.zero_grad()
     10         loss.backward()
     11         optimizer.step()
     12
     13         avg_cost += loss / total_batch
     14
     15     print('Epoch: %d Loss = %f' %(epoch+1, avg_cost))
     16 print('Learning finished')
```

Vanishing Gradient 문제 해결



Epoch: 1	Loss =	2.308627
Epoch: 2	Loss =	2.304830
Epoch: 3	Loss =	2.297726
Epoch: 4	Loss =	2.240335
Epoch: 5	Loss =	1.776216
Epoch: 6	Loss =	1.274918
Epoch: 7	Loss =	0.842344
Epoch: 8	Loss =	0.654143
Epoch: 9	Loss =	0.546259
Epoch: 10	Loss =	0.472251
Epoch: 11	Loss =	0.417811
Epoch: 12	Loss =	0.373456
Epoch: 13	Loss =	0.335884
Epoch: 14	Loss =	0.304409
Epoch: 15	Loss =	0.273746
Learning finished		

Vanishing Gradient 문제 실습

▪ Vanishing Gradient 문제 해결(2): 가중치 초기화 사용

- (4) Weight parameter 저장 및 불러오기
- (5) MNIST Test dataset 분류 성능 확인

```
[35] 1 torch.save(network.state_dict(), parameterPath+"fmlp_mnist.pth")
```

4

```
[36] 1 new_network = FMLP()  
    2 new_network.load_state_dict(torch.load(parameterPath+"fmlp_mnist.pth"))
```

<All keys matched successfully>

```
[37] 1 with torch.no_grad(): # test에서는 기울기 계산 제외  
    2  
    3 img_test = mnist_test.data.float()  
    4 label_test = mnist_test.targets  
    5  
    6 prediction = network(img_test) # 전체 test data를 한번에 계산  
    7  
    8 correct_prediction = torch.argmax(prediction, 1) == label_test  
    9 accuracy = correct_prediction.float().mean()  
   10 print("Accuracy:", accuracy.item())
```

5

Accuracy: 0.8885999917984009

정답률: 88.5%

Questions & Answers

Dongsan Jun (dsjun@dau.ac.kr)

Image Signal Processing Laboratory (www.donga-ispl.kr)

Division of Computer·AI Engineering

Dong-A University, Busan, Rep. of Korea